A
Project Report
On
"Speech Emotion Recognition using LSTM"

Prepared by:
23AIML056, 23AIML064, 23AIML074

A Report Submitted to
Charotar University of Science and Technology
for Partial Fulfilment of the Requirements for the
Degree of Bachelor of Technology
in Information Technology
(4th Semester Project I – AIML211)

Submitted at

CHARUSAT®
अमृतं तु विद्या
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
Chandubhai S. Patel Institute of Technology
At: Changa, Dist.: Anand – 388421
April 2025

Kavya Prajapati
Artificial Intelligence and Machine Learning, Chandubhai S Patel Institute of Technology Charotar University of Science and Technology, Changa, INDIA
23aiml056@charusat.edu.in

Aarya Shah
Artificial Intelligence and Machine Learning, Chandubhai S Patel Institute of Technology Charotar University of Science and Technology, Changa, INDIA
23aiml064@charusat.edu.in

Vansh Mehta
Artificial Intelligence and Machine Learning Chandubhai S Patel Institute of Technology Charotar University of Science and Technology, Changa, INDIA
23aiml074@charusat.edu.in

Abstract

In this work, we conduct an extensive comparison of various approaches to speech based emotion recognition systems. The analyses were carried out on audio recordings from Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS). After pre-processing the raw audio files, features such as Mel Spectrogram, Mel-Frequency Cepstral Coefficients (MFCCs),Chroma stft, pitch and energy were considered. The significance of these features for emotion classification was compared by applying methods such as Long Short Term Memory (LSTM), Recurrent Neural Networks (RNNs) On the 7-class classification task, an accuracy of 82% was achieved with a 4-layer 2 dimensional RNN using the Mel Spectrogram,mfcc and chroma stft features. We also observe that, in emotion recognition, the choice of audio features impacts the results much more than the model complexity

Introduction:

Humans use emotions to express how they feel, and most of the time, we do this through our voice. With the growth of technology, machines are now being trained to understand these emotions too. This process is called **Speech Emotion Recognition (SER)**. SER helps computers recognize the feelings in someone's voice — like happiness, anger, sadness, or fear — and respond in a more human-like way.

Speech emotion recognition is useful in many areas, such as in cars to detect if a driver is stressed, in health apps to track emotional well-being, or in smart assistants to improve user experience. To make this possible, we need to extract important features from the speech. Some commonly used features are:

- **MFCC (Mel Frequency Cepstral Coefficients)** – helps in understanding how humans hear sound.
- **Mel Spectrogram** – shows how energy in different frequencies changes over time.
- **Chroma** – represents the pitch and harmony in the speech.
- **STFT (Short-Time Fourier Transform)** – breaks the speech into smaller parts to study how sound changes over time.

Once these features are extracted, a machine learning model is used to recognize the emotions. In this project, we use **Recurrent Neural Networks (RNN)** because they are good at working with time-based data like speech. RNNs can remember patterns in speech, which helps in better emotion recognition.

TECHNOLOGIES:

Backend:

Python: It is the core programming language throughout the project.

Pandas: it is a library for data management and analysis in Python.

NumPy: it is a package for scientific computing and operations related to tensors in Python.

Matplotlib: it is a plotting library used for creating visualizations in Python.

Keras: A high-level neural networks API, serving as an interface for constructing and training deep learning models, built on top of TensorFlow.

TensorFlow: An open-source machine learning framework developed by Google, widely used for building and deploying machine learning and deep learning models.

PyTorch: A Python library offering tools for tensor computation and dynamic computational graph creation, commonly employed for machine learning and deep learning tasks.

Scikit-learn: A comprehensive machine learning library featuring a variety of algorithms for classification, regression, clustering, dimensionality reduction, and more.

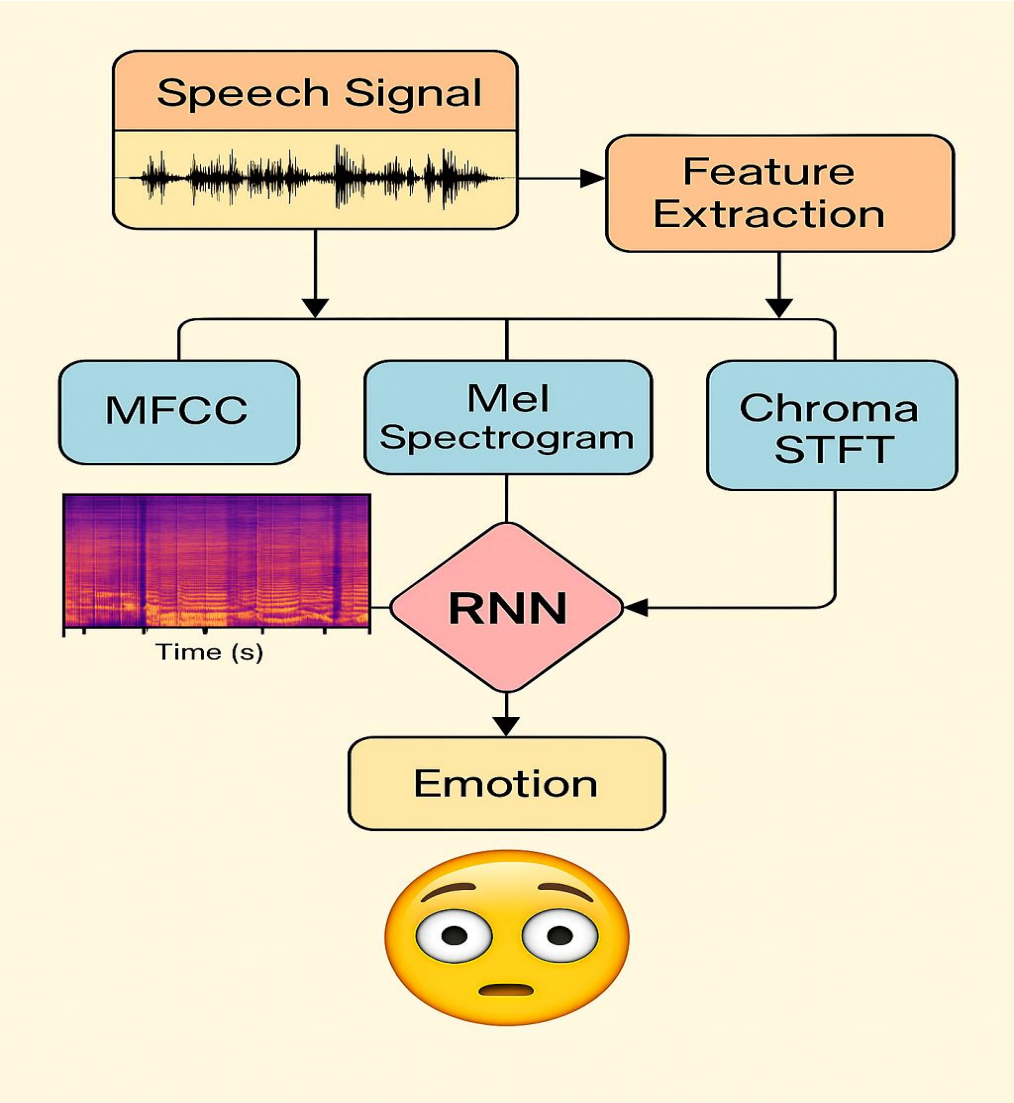
IPython: IPython is an interactive command-line interface for Python that enhances productivity and ease of use, especially for data science and machine learning workflows**.**

Joblib: It is a Python library designed for efficient computation, particularly in data science and machine learning workflows. Its main use is Parallel Processing.

Frontend:

Tkinter: It is a python GUI library that is used build the desktop based graphical user interface. In this project Tkinter is used for taking audio input from the user displaying prediction result.

WORKFLOW EXPLANATION:



---

1. Data Acquisition

The system starts by accessing the RAVDESS (Ryerson Audio-Visual Database of Emotional Speech and Song) dataset. This dataset contains audio clips of human speech, each encoded with specific emotional expressions such as happiness, sadness, fear, anger, and calmness.

- The directory is traversed using Python's os.listdir() and os.path.isdir() functions.
- Each audio file's filename includes metadata about the speaker and the emotion.
- From the filenames, the emotion code (e.g., 03 for happy) is extracted.

The corresponding audio file paths and emotion codes are collected in lists.

---

2. Emotion Label Mapping and Dataset Construction

Once the raw file paths and emotion codes are extracted:

- A dictionary is defined to map numeric emotion codes to human-readable emotion labels:
  - Example: 1 becomes 'neutral', 3 becomes 'happy', etc.
- These mapped labels and their corresponding audio paths are merged into a pandas.DataFrame.

The resulting Dataframe (Ravdess_df) provides structured access to the emotion and file path for each audio sample.

---

3. Feature Extraction

For each audio file in the dataset, a set of acoustic features is extracted using the librosa library. These features capture both spectral and temporal characteristics of the audio.

- The extraction is handled using a parallel processing approach via joblib.Parallel to speed up computation.
- Three main types of features are extracted:
  1. **MFCC (Mel Frequency Cepstral Coefficients):** Represents short-term power spectrum of sound, helpful for capturing the timbral aspect.
  2. **Chroma_STFT(Chroma Short-Time Fourier Transform):** Combines chromagram and STFT to represent harmonic content. The chroma vector expresses pitch class distribution.
  3. **Mel Spectrogram:** Spectral representation using the Mel scale which aligns with human auditory perception.

Each feature type is time-averaged and concatenated to form a unified feature vector per audio sample.

---

4. Dataset Preparation

After all features are extracted:

- The features are stored in a new Dataframe column and checked for None or missing values.
- The dataset is filtered to retain only the audio files with successfully extracted features.
- These feature vectors are collected into a NumPy array X.
- The X array is expanded along the last axis to make it suitable for LSTM input. The resulting shape is (samples, features, 1).

The labels (y) are also processed:

- The emotion labels are one-hot encoded using sklearn.preprocessing.OneHotEncoder to convert them into a binary matrix format.

Example: 'happy' becomes [0, 0, 1, 0, 0, 0, 0, 0].

---

5. Data Splitting

- The complete dataset is split into training and testing sets using train_test_split() with an 80-20 split.
- X_train, X_test, y_train, and y_test are generated for training and evaluation.

---

6. Model Building

A deep learning model is constructed using Keras Sequential API. The model is a stack of layers, including:

- **LSTM Layer**:
  - Contains 256 units.
  - Captures temporal dependencies in the acoustic features across time.
  - Input shape is (40, 1), assuming each sample has 40 feature components.
- **Dropout Layers**:
  - Dropout rate of 0.2 is used.
  - These layers prevent overfitting by randomly dropping some neurons during training.
- **Dense Layers**:
  - One dense layer with 128 units and ReLU activation.
  - Another dense layer with 64 units and ReLU activation.
  - These layers help in non-linear transformation and classification.
- **Output Layer**:
  - Dense layer with 8 units (for 8 emotion classes) and Softmax activation.
  - Outputs a probability distribution across all emotion classes.

The model is compiled with:

- Loss function: 'categorical_crossentropy' for multi-class classification.
- Optimizer: 'adam'
- Metric: 'accuracy'

---

7. Model Training

- The model is trained on the training dataset using the .fit() method.
- Training settings include:
  - epochs=300
  - batch_size=64
  - validation_split=0.2

During training, the model's performance is monitored on a validation set.

---

8. Model Evaluation

After training, the model's performance is evaluated using:

```
model.evaluate(X_test, y_test)
model.evaluate(X_train, y_train)
```

This provides accuracy and loss values for both training and testing data, helping assess overfitting or underfitting.

---

9. Audio Visualization (Waveplot and Spectrogram)

Before or during data exploration, the audio signals are visualized using:

- **Waveplots**: Time-domain representation showing amplitude variations over time.
- **Spectrograms**: Frequency-domain representations showing intensity at different frequencies over time.

These visualizations are generated using librosa.display.waveshow() and librosa.display.specshow() respectively.

---

10. Prediction and Inference

Once the model is trained and validated:

- New/unseen audio files can be fed into the same feature extraction pipeline.
- The resulting feature vector is passed through the model for inference.
- The predicted class represents the detected emotion, e.g., 'angry', 'happy', etc.

This output is integrated into real-time systems like emotion-aware virtual agents, mental health assessment tools, or customer sentiment monitoring platforms.

FRONT-END:

The frontend used in the project is designed using Tkinter, a standard GUI library of python. Tinkter provides a fast and easy way to build simple desktop applications.

- **User Input (Audio Recording)**:
  A button-based system is implemented to allow users to record their voice through a microphone. This makes the model more interactive and real-time.
- **Emotion Prediction Trigger:**
  Once the audio is recorded, the user can submit the recording via another button. This sends the audio to the backend where features are extracted and emotion is predicted.

- **Result Display:**
  The predicted emotion is displayed directly on the GUI, giving the user immediate feedback.
- **Error Handling:**
  The front-end handles errors such as missing input or improper recording by showing simple error messages, enhancing user experience.
- **Design Simplicity:**
  The interface is kept minimalistic to ensure smooth performance and accessibility, even for users unfamiliar with technology.

BACK-END:

The backend of this project is built entirely using Python, leveraging several powerful libraries and frameworks to handle the core functionalities such as audio processing, feature extraction, model training, and prediction. The backend operates behind the scenes and is responsible for all data processing and logic execution that drives the system.

Key Components of the Backend:

1. **Audio Processing and Feature Extraction:**
   - Librosa is used to extract features from raw audio such as:
     - MFCC (Mel-Frequency Cepstral Coefficients)
     - Chroma STFT (Short-Time Fourier Transform)
     - Mel Spectrogram
   - These features represent the spectral and temporal characteristics of the audio, making them suitable for emotion classification.
2. **Data Handling:**
   - Pandas is used for organizing data into structured DataFrames.
   - NumPy is used for numerical computations and matrix operations.
3. **Machine Learning and Deep Learning:**
   - Keras (with TensorFlow backend) is used to create a deep learning model.
     - The model architecture includes:
       - A LSTM (Long Short-Term Memory) layer to handle time-series data.
       - Dense layers with ReLU and softmax activation for emotion classification.
       - Dropout layers to prevent overfitting.
     - The model is compiled with the Adam optimizer and categorical crossentropy loss function.
   - Scikit-learn is used for:

- One-hot encoding the emotion labels
- Splitting the dataset (train-test split)
- Evaluation

4. **Parallel Processing:**
    - Joblib is used to process feature extraction tasks in parallel, significantly speeding up the data preparation phase.
5. **Development & Testing:**
    - Jupyter Notebook is used for development, visualization, experimentation, and model debugging.

Workflow Summary:

- Raw audio input is received and processed to extract relevant features.
- The extracted features are reshaped and passed into the trained LSTM model.
- The model outputs the predicted emotion class, which is then sent back to the GUI for display.

Advantages of this Backend Design:

- Modular and scalable structure
- Integration of high-performance ML/DL libraries
- Real-time processing capabilities
- Compatible with GUI-based and future web-based

Speech Emotion Recognition

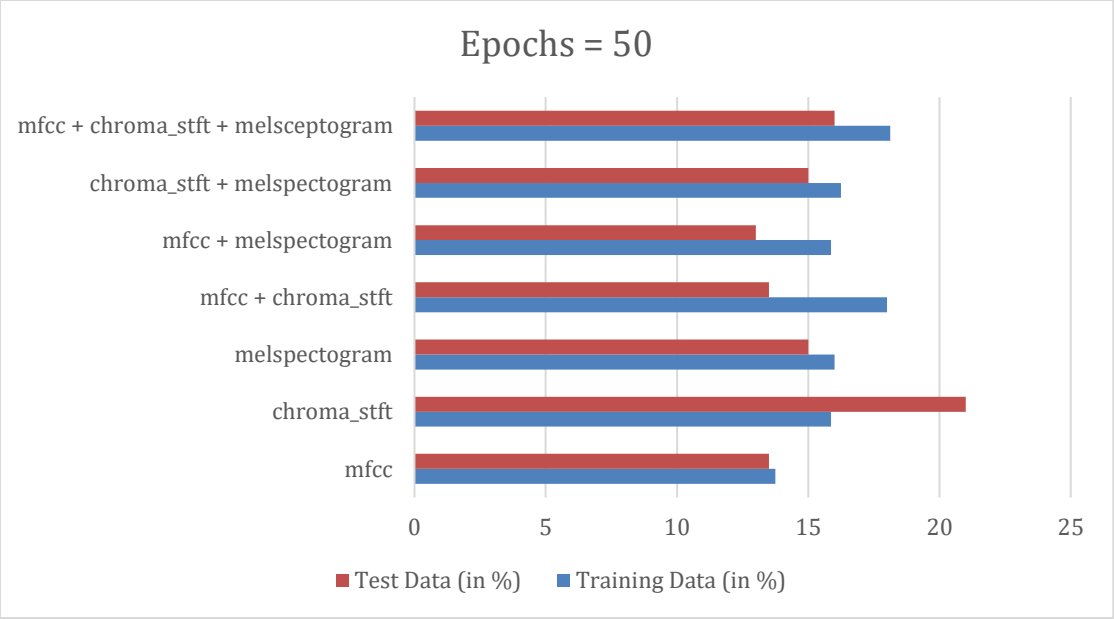
Model Accuracy using different Feature Extractions and epochs

mfcc - Mel Frequency Cepstral Coefficient
chroma_stft – Chroma Short-Time Fourier Transform
melspectogram

Below are the observations recorded during the training and testing of the model on the ravdess dataset

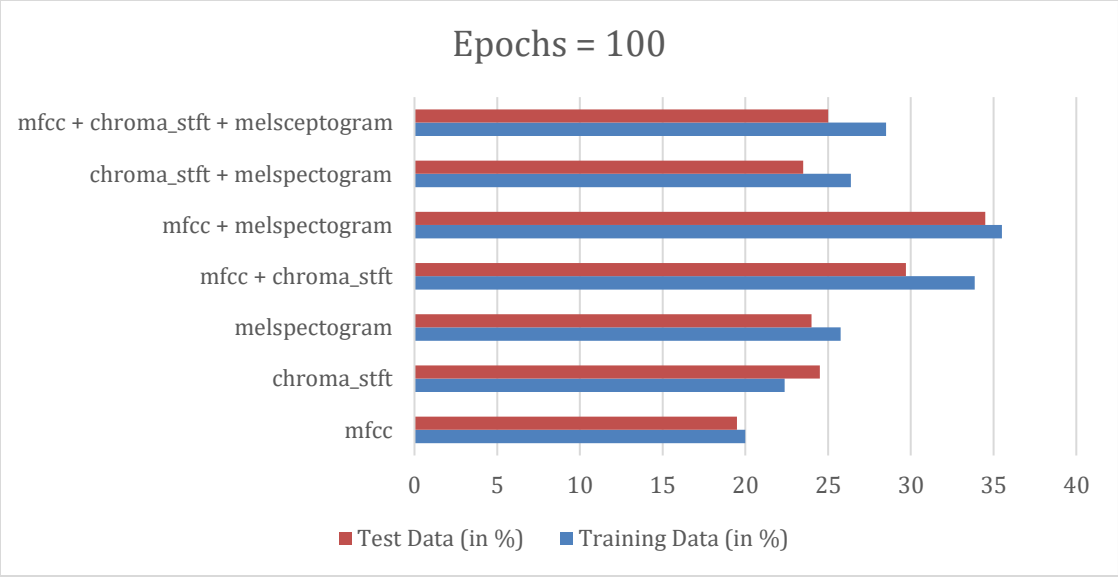
Epochs = 50

Method	Training Data (in %)	Test Data (in %)
mfcc	13.75	13.5
chroma_stft	15.87	21
melspectogram	16	15
mfcc + chroma_stft	18	13.5
mfcc + melspectogram	15.87	13
chroma_stft + melspectogram	16.25	15
mfcc + chroma_stft + melsceptogram	18.13	16



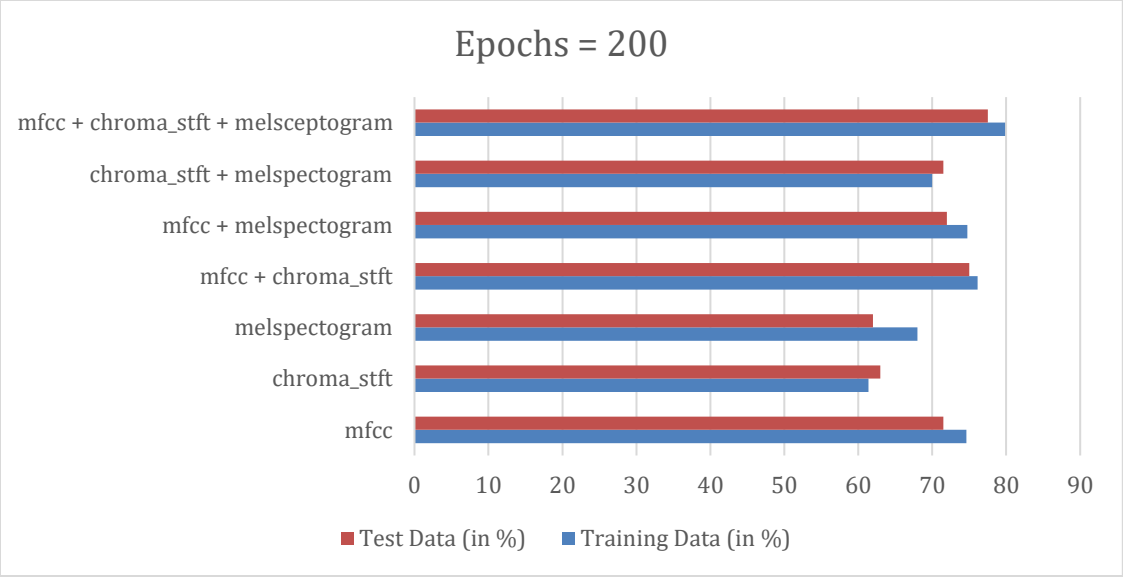
Epochs = 100

Method	Training Data (in %)	Test Data (in %)
mfcc	20	19.5
chroma_stft	22.37	24.5
melspectogram	25.75	24
mfcc + chroma_stft	33.86	29.71
mfcc + melspectogram	35.5	34.5
chroma_stft + melspectogram	26.37	23.5
mfcc + chroma_stft + melsceptogram	28.5	25



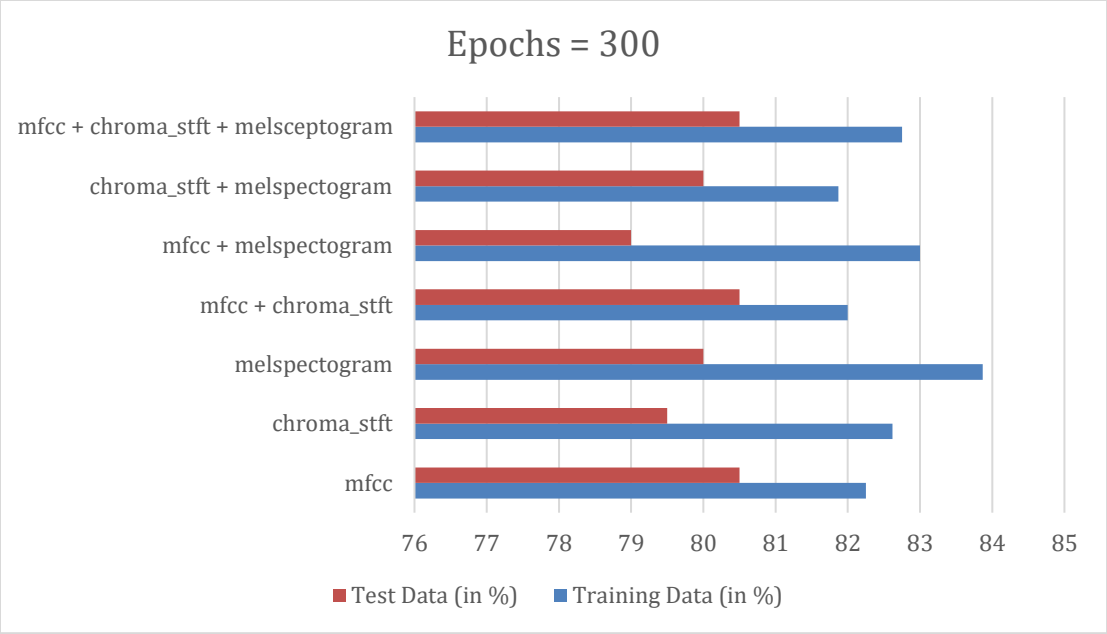
Epochs = 200

Method	Training Data (in %)	Test Data (in %)
mfcc	74.62	71.5
chroma_stft	61.37	63
melspectogram	68	62
mfcc + chroma_stft	76.13	75
mfcc + melspectogram	74.75	72
chroma_stft + melspectogram	70	71.5
mfcc + chroma_stft + melsceptogram	79.87	77.5



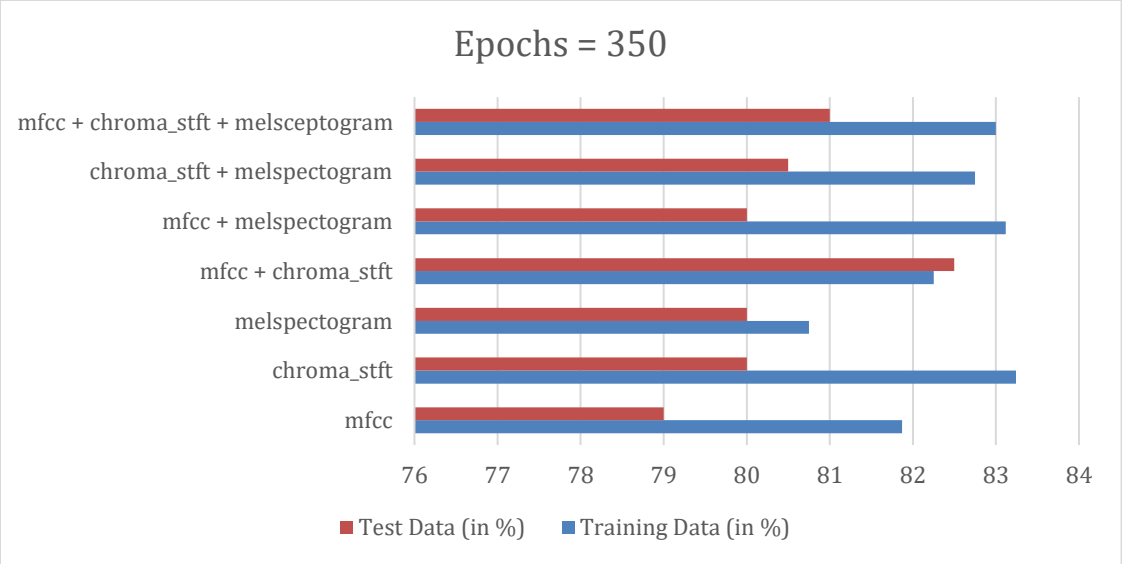
Epochs = 300

Method	Training Data (in %)	Test Data (in %)
mfcc	82.25	80.5
chroma_stft	82.62	79.5
melspectogram	83.87	80
mfcc + chroma_stft	82	80.5
mfcc + melspectogram	83	79
chroma_stft + melspectogram	81.87	80
mfcc + chroma_stft + melsceptogram	82.75	80.5



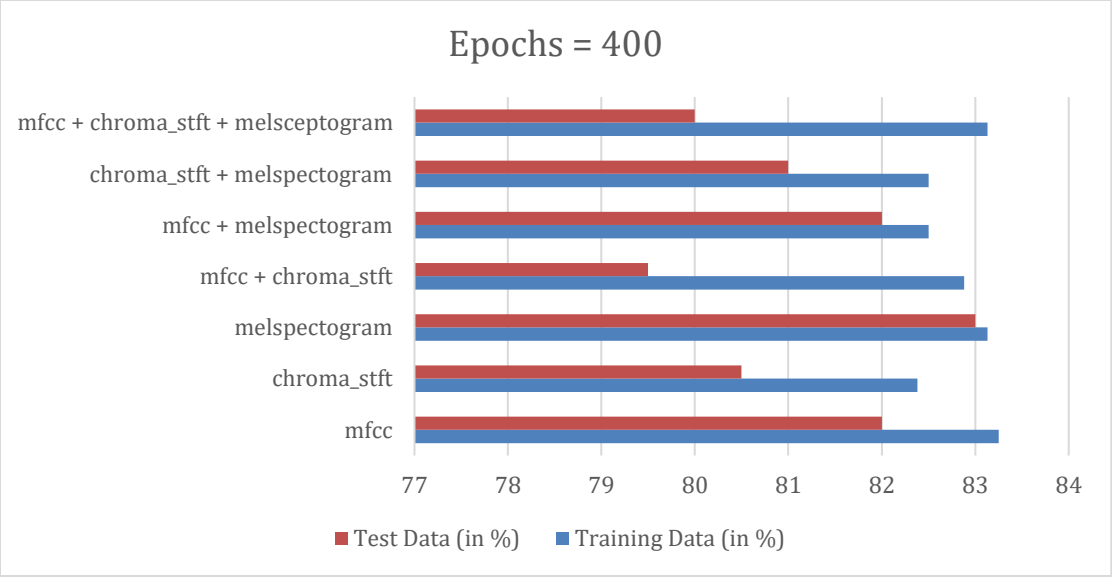
Epochs = 350

Method	Training Data (in %)	Test Data (in %)
mfcc	81.87	79
chroma_stft	83.24	80
melspectogram	80.75	80
mfcc + chroma_stft	82.25	82.5
mfcc + melspectogram	83.12	80
chroma_stft + melspectogram	82.75	80.5
mfcc + chroma_stft + melsceptogram	83	81



Epochs = 400

Method	Training Data (in %)	Test Data (in %)
mfcc	83.25	82
chroma_stft	82.38	80.5
melspectogram	83.13	83
mfcc + chroma_stft	82.88	79.5
mfcc + melspectogram	82.5	82
chroma_stft + melspectogram	82.5	81
mfcc + chroma_stft + melsceptogram	83.13	80



Appendix

A.1 Weekly Progress Summaries

- **Week 1 (02/01/2025 - 09/01/2025)**: Finding and learning prerequisites like python library Librosa.
- **Week 2 (06/02/2025 - 13/02/2025)**: Preprocessing the data
- **Week 3 (13/02/2025 - 20/02/2025)**: Feature extraction using mfcc,chroma stft,mel spectogram.
- **Week 4 (20/02/2025 - 27/02/2025)**: Training Long Short Term Memory Recurrent neural network based model on the data features extracted by doing EDA.
- **Week 5 (27/02/2025 - 06/03/2025)**: Analyzing the model with variety of different inputs and varying parameters.
- **Week 6-7 (06/03/2025 - 14/03/2025)**: Adding real time emotion prediction with graphical user interface.

A.2 SWOT Analysis

- **Strengths**: Accurately detects emotions from speech using deep learning and audio features.
- **Weaknesses**: May some time confuses between some emotion which possess same frequency ,pitch, amplitude.
- **Opportunities**: Can be expanded to web/mobile platforms and real-world applications.
- **Threats**: Privacy concerns for voice data collection in real-time usage.